



Si algo falla

Guía 6 · Taller 02 de caso aplicado de IA en industria

Trece cosas se rompieron montando este laboratorio, seis solo en Windows. Ninguna es interesante por sí sola; juntas son la mitad del tiempo. Están acá para que no te cuesten a ti.

Lo primero, siempre

```
make verificar
```

Corre 24 comprobaciones sobre todo el laboratorio y termina diciendo cuáles pasaron. Si algo falla, la línea sale con `x` y debajo el detalle con el número que salió — no «falló», sino qué se midió y qué se esperaba.

Para un reto solo: `make verificar ARGS="--reto 2"`.

Docker

«Cannot connect to the Docker daemon». Docker Desktop no está corriendo, o todavía está arrancando. Ábrelo, espera a que el icono deje de moverse.

«No space left on device» a mitad de la construcción. Las imágenes ocupan unos 6 GB y la caché de construcción otro tanto.

```
docker builder prune -f
```

Deja 12 GB libres antes de empezar.

Cambiaste un archivo y no pasa nada. Un 404 en una ruta que está en el archivo. Una interfaz que revienta por una columna. Un `git pull` que parece no haber hecho nada. Son tres síntomas del mismo error: **el contenedor sirve el código con el que arrancó**, no el que hay en disco.

```
docker compose restart plataforma
```

La pregunta que resuelve esto en cualquier laboratorio con volúmenes montados: **¿el proceso vio este archivo?**

Windows

`make` no existe. Windows no lo trae. Usa el script equivalente:

```
.\taller.ps1 arriba
.\taller.ps1 romper sesgo_silencioso
```

«No se puede cargar el archivo ... porque la ejecución de scripts está deshabilitada». Es la política de PowerShell:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Con `-Scope Process` solo afecta a esta ventana, que es lo que quieres.

Los finales de línea. Git puede convertir los saltos al clonar y dejar los scripts con `CRLF`, que dentro de un contenedor Linux no arrancan. Si ves errores raros de sintaxis en un `.sh`:

```
git config --global core.autocrlf input
```

y vuelve a clonar.

El agente

No tengo llave de LLM. El taller corre igual:

```
PROVEEDOR_LLM=mock
```

en tu `.env`. Recorre el bucle y el grafo enteros, así que sirve para comprobar el cableado. No razona — el diagnóstico sale con texto de relleno.

Las llaves gratuitas. `google` (aistudio.google.com) y `groq` (console.groq.com) tienen nivel gratuito y funcionan bien para esto. Se elige en `.env` con `PROVEEDOR_LLM` y `MODELO_LLM`.

El agente no llamó ninguna herramienta. Si estás construyendo el grafo tú, es casi seguro esto:

```
ToolNode(HERRAMIENTAS, messages_key="mensajes")
```

`ToolNode` busca una clave `messages` por omisión; nuestro estado la llama `mensajes`. Sin ese parámetro no encuentra nada que ejecutar, **no lanza ninguna excepción** y pasa de largo. El diagnóstico sale escrito sobre la nada, y se lee convincente.

Los datos

Veo 17,304 filas en vez de 17,472. Quedaste en `feed_caído` : falta una tienda. `make reparar` .

Mis números no son los de la guía. El mundo se genera contra la fecha del día, así que el conteo de modelos sobre umbral y las unidades se mueven entre ejecuciones. El MAPE y el sesgo, en cambio, coinciden al primer decimal.

Quiero empezar de cero. `make reset` borra todo, incluidos los 192 modelos, y después hay que volver a correr `make seed` . Son unos minutos.

MÓDULO III · Herramientas para la Inteligencia Artificial Generativa y aplicaciones para la industria

Taller 02 de caso aplicado de IA en industria